

Motion Detection Alarm

IoT Prototype using Arduino Uno and a PIR Sensor

Internship Task 3 | Prepared by: Praharsha | October 2026

1. Objective

To build a low-cost motion detection alarm with an Arduino Uno. A PIR (passive infrared) sensor detects movement of a person in its field of view; the Arduino then switches on a buzzer and an LED and logs each event, with its time, on the serial monitor. The project follows the standard IoT pattern of sensing, processing, acting and reporting, and can be extended to send alerts over Wi-Fi.

2. Components Used

Component	Qty	Purpose
Arduino Uno R3 with USB cable	1	Controller and power source
HC-SR501 PIR motion sensor	1	Detects infrared change caused by a moving body
Active buzzer (5 V)	1	Audible alarm
LED (red)	1	Visual alarm
Resistor, 220 ohm	1	Limits LED current
Breadboard and jumper wires	-	Connections

3. Circuit Connections

From	To
PIR VCC	Arduino 5V
PIR GND	Arduino GND
PIR OUT	Arduino D2
Buzzer positive (+)	Arduino D8
Buzzer negative (-)	Arduino GND
LED anode (longer leg)	220 ohm resistor, then Arduino D9
LED cathode (shorter leg)	Arduino GND

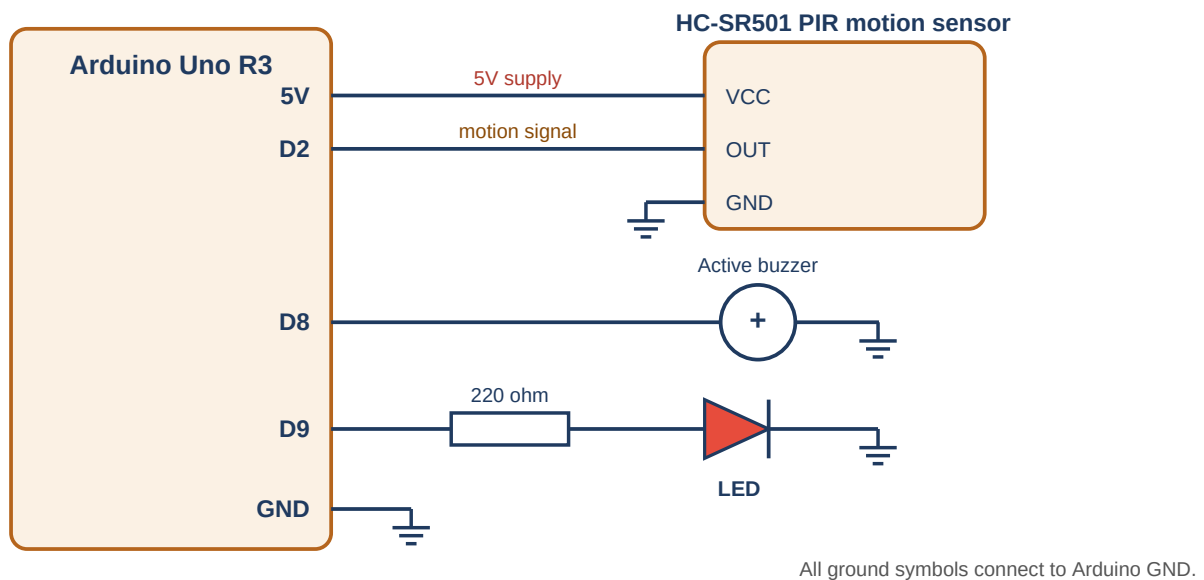


Figure 1. Circuit diagram of the motion detection alarm.

4. Working Principle

Every warm body emits infrared radiation. The PIR sensor contains a pyroelectric element that reacts to changes in the infrared pattern, so a person walking across its field of view makes its output pin go HIGH. The Arduino reads that pin on D2. While the pin is HIGH, the program turns the LED on, beeps the buzzer in a repeating on-off pattern and prints one log line for the event. When the pin returns to LOW, the alarm stops and the program prints that the area is clear.

The HC-SR501 needs roughly half a minute to stabilise after power-up, so the sketch waits 30 seconds before arming. The two small potentiometers on the sensor board adjust detection sensitivity and how long the output stays HIGH.

5. Arduino Code

```
// Motion Detection Alarm - Arduino Uno + HC-SR501 PIR sensor
const int pirPin = 2;           // PIR output pin
const int buzzerPin = 8;        // active buzzer
const int ledPin = 9;           // alarm LED (through 220 ohm resistor)

bool motionActive = false;      // remembers if motion is already being reported
unsigned int eventCount = 0;     // number of motion events since start-up

void setup() {
  pinMode(pirPin, INPUT);
  pinMode(buzzerPin, OUTPUT);
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
  Serial.println("Warming up PIR sensor (30 s)...");
  delay(30000);                 // PIR needs time to settle after power-up
  Serial.println("System armed. Waiting for motion...");
}

void loop() {
  if (digitalRead(pirPin) == HIGH) {      // motion present
    if (!motionActive) {                  // log only once per event
      motionActive = true;
      eventCount++;
      Serial.print("MOTION DETECTED! Event #");
      Serial.print(eventCount);
      Serial.print(" at ");
      Serial.print(millis() / 1000);
      Serial.println(" s");
    }
    digitalWrite(ledPin, HIGH);           // alarm LED on
    digitalWrite(buzzerPin, HIGH);        // beep
    delay(200);
    digitalWrite(buzzerPin, LOW);
    delay(200);
  } else {                                // no motion
    if (motionActive) {
      motionActive = false;
      Serial.println("Area clear.");
    }
    digitalWrite(ledPin, LOW);
    digitalWrite(buzzerPin, LOW);
    delay(50);
  }
}
```

Listing 1. Complete sketch.

6. Code Explanation

Part of the code	What it does
<code>pirPin, buzzerPin, ledPin</code>	Named constants for the three pins, so wiring changes need only one edit.
<code>motionActive, eventCount</code>	A flag that stops the same event being logged repeatedly, and a counter of events since start-up.
<code>pinMode(...)</code>	Sets D2 as an input (sensor) and D8, D9 as outputs (buzzer, LED).
<code>delay(30000)</code>	Warm-up period so the PIR can settle and does not give false alarms at power-on.
<code>digitalRead(pirPin) == HIGH</code>	Checks whether the sensor currently reports motion.
<code>if (!motionActive)</code>	Runs only on the first loop of a new event: sets the flag, increments the counter and prints the time from <code>millis()</code> .
<code>digitalWrite(..., HIGH/LOW)</code>	Turns the LED on and pulses the buzzer 200 ms on, 200 ms off while motion lasts.
<code>else { ... }</code>	When motion stops, prints "Area clear." once and switches both outputs off.

7. Testing Procedure

1. Connect the circuit as in Section 3 and upload the sketch from the Arduino IDE (board: Arduino Uno).
2. Open the Serial Monitor at 9600 baud and wait for the "System armed" message.
3. Walk or wave a hand in front of the PIR dome: the LED should light, the buzzer should beep and an event line should appear.
4. Stand still or move away: after the sensor's hold time the output drops, the alarm stops and "Area clear." is printed.
5. Adjust the sensitivity and time-delay potentiometers and repeat to see the effect.

8. Expected Output

The serial monitor should show lines in the following form. The times depend on when you trigger the sensor, so the values below are only an example.

```
Warming up PIR sensor (30 s)...  
System armed. Waiting for motion...  
MOTION DETECTED! Event #1 at 42 s  
Area clear.  
MOTION DETECTED! Event #2 at 58 s  
Area clear.
```

Listing 2. Example serial output.

9. Output Images

Add photos of your own build and screenshots of the serial monitor over the boxes below.

Figure 2: Wired prototype

Photo of the Arduino, PIR, buzzer and LED on the breadboard

[paste your photo / screenshot here]

Figure 3: Idle state

No motion: LED off, buzzer silent

[paste your photo / screenshot here]

Figure 4: Motion detected

Hand in front of the PIR: LED on, buzzer active

[paste your photo / screenshot here]

Figure 5: Serial Monitor log

Screenshot showing detection and clear messages

[paste your photo / screenshot here]

10. Troubleshooting

Problem	Likely cause and fix
Alarm triggers at power-on	The PIR is still warming up. Wait for the 30 s start-up message.
Frequent false alarms	Lower the sensitivity potentiometer and keep the sensor away from heaters, sunlight and fans.
Nothing happens	Check 5V and GND on the PIR and confirm OUT goes to D2.
Buzzer is silent	Check polarity (+ to D8). A passive buzzer will not sound with this code; use an active one.

11. Applications and Extensions

The same design suits room or cupboard intrusion alerts, entry detection for shops and labs, and automatic lighting. To make it a fuller IoT system, replace the Uno with an ESP32 or ESP8266 and send each event to a phone using MQTT, a cloud dashboard or a messaging bot. An ultrasonic or door-contact sensor can be added for more reliable detection.

12. Conclusion

The prototype detects motion with a PIR sensor and responds with a local alarm and a timestamped log, using only a few inexpensive parts and no external libraries. It demonstrates the sense, process, act and report cycle and provides a base that can be connected to the internet with a Wi-Fi microcontroller.